הרצאה 1: הקדמה

הקורס שלפניכם עוסק בתורת האלגוריתמים המתמטית. במהלך הקורס:

1. נגדיר את המושגים הבסיסיים הנדרשים לעיסוק בתחום.
2. נלמד מספר פרדיגמות (שיטות) לפיתוח אלגוריתמים.
3. נלמד מספר רב של אלגוריתמים בסיסיים.

אזהרה

בטרם ניגש לחומר ברצוני להזהיר את הקורא:

לצערנו לא קיימת שיטה המבטיחה הצלחה בכל ניסיון לפתח אלגוריתמים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

כל בעיה חישובית מהווה אתגר חדש. יש אתגרים קלים, יש אתגרים קשים למדי, וקיימות גם בעיות שטרם נפתרו. החומר שנלמד בקורס, ייתן בידכם מכשירים לפיתוח אלגוריתמים ולהערכת נכונותם וסיבוכיותם, אך האתגר היצירתי יהיה נוכח בכל משימה אלגוריתמית שתיקחו על עצמכם, וזה גם היופי בתורת האלגוריתמים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

המושגים הבסיסיים

כל תורה מתמטית מתחילה בהצגת המושגים הבסיסיים בהם היא משתמשת וזוהי מטרת ההרצאה שלנו.

המושגים הבסיסיים שעלינו להכיר ולהבין הם:

1. בעיה חישובית.
2. אלגוריתם
3. נכונות של אלגוריתם.
4. סיבוכיות של אלגוריתם.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הבעיה החישובית

בעיה חישובית היא בעיה שניתנת לפתרון על ידי אלגוריתם, או תכנית מחשב.

אנו נגדיר בעיה חישוביות ללא תלות באלגוריתם הפותר את הבעיה. בדרך זו נמנע ממצב בו המושג "בעיה חישובית" מוגדר על ידי המושג "אלגוריתם", ואחר כך "אלגוריתם" מוגדר על ידי "בעיה חישובית".

אפשר לתאר בעיה חישובית על ידי טבלה אינסופית של זוגות. האיבר הראשון בכל זוג הוא אחד הקלטים לבעיה והאיבר השני הוא הפלט המתאים לאיבר הראשון.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה - הבעיה החישובית "סכום של מערך"

קבוצת הקלטים

קבוצת הקלטים מכילה את כל מערכי המספרים. (בכל אורך סופי).

שימו לב:

קבוצת הקלטים היא אינסופית אך כל איבר שלה (מערך מספרים) הוא סופי.

הפלט

לכל מערך מספרים A ובו n איברים, מתאים הפלט

$$Out(A) = A[1] + \cdots + A[n]$$

באיור 1.1 מוצגת טבלה ובה חלק קטן מהגדרת הבעיה **סכום של מערך**.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמא - הבעיה החישובית ״סכום של מערך״

הקלט	הפלט המתאים
$[7]$	7
$[3,-2]$	1
$[12,-4,5,9]$	22
.	.
.	.
.	.

איור 1.1: תיאור של הבעיה החישובית ״סכום של מערך מספרים״

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הטבלה מכילה שתי עמודות: בכל תא בעמודה הימנית מוצג מערך מספרים המשתייך לקבוצת הקלטים, הקלטים המופיעים בטבלה נבחרו באופן שרירותי. בעמודה השנייה מוצג מול כל קלט, הפלט המתאים לו.

ההגדרה הפורמלית של ״סכום של מערכי מספרים״ מוצגת באיור 1.2:

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הקלט – *In*

כל מערכי המספרים (שארכם סופי).

הפלט - *Out*

לכל מערך מספרים A ובו n איברים, מתאים הפלט:

$$Out(A) = A[1] + \ldots + A[n] .$$

איור 1.2: הגדרת "סכום של מערך"

בטרם תמשיכו בקריאת המצגת, נסו להגדיר את הבעיות החישוביות הבאות:

1. איבר מכסימלי במערך.
2. ממוצע של מערך.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה פורמלית של בעיה חישובית

נגדיר בעיה חישובית באמצעות שני מרכיבים:

1. קבוצת הקלטים.
2. תיאור הפלט.

קבוצת הקלטים – בהגדרה נכתוב "הקלט"

זוהי קבוצה המכילה את כל הקלטים של הבעיה. לכל בעיה חישובית P, נסמן את קבוצת הקלטים של P ב-In_p.

שימו לב: בדרך כלל קבוצת הקלטים היא אינסופית.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תיאור הפלט – בהגדרה נכתוב

"הפלט"

לכל קלט $i \in In_p$ מתאים פלט שנסמנו ב-$Out_P(i)$.

סימון

הבעיה החישובית P תסומן על ידי

$$P = \langle In_P, out_P \rangle$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרת "אלגוריתם"

אלגוריתם הוא תיאור של שיטת עבודה הניתנת לביצוע על ידי תכנה, לפתרון בעיה חישובית (שכבר הגדרנו).
כל אלגוריתם ניתן לתיאור באמצעות סדרת הוראות, אך לא כל סדרת הוראות מהווה אלגוריתם. אנו נגדיר אלגוריתם ביחס לבעיה חישובית נתונה. ההגדרה הפורמלית של אלגוריתם לפתרון P מוצגת באיור 1.3

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 1.3: הגדרת אלגוריתם לפתרון בעיה חישובית P

אלגוריתם הפותר בעיה חישובית $P = \langle In_P, Out_P \rangle$, הוא סדרת הוראות (שיטה) Al, אשר מקבלת כקלט כל איבר $i \in In_p$, מסיימת את החישוב, ומחזירה את הערך $Out_p(i)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תיאור אלגוריתמים – קוד דמה

בבואנו לתאר אלגוריתם, עלינו להעדיף את הבהירות הנדרשת להבנה אנושית, על פני הדיוק הטכני הנדרש מתכנה ניתנת לביצוע.

הדרך שנבחרה לתיאור אלגוריתמים היא שימוש בקוד דמה (Pseudo Code).

קוד דמה, הוא "שפה" הדומה למדי לשפת תכנות, אך משמיטה את הפרטים הטכניים, הנחוצים למהדר (Compiler), אך מקשים על ההבנה האנושית.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

יסודות קוד דמה

אלגוריתם המתואר באמצעות קוד דמה הוא סדרה של הוראות, כאשר כל הוראה מבוטאת באופן חפשי, על ידי ערוב של כתיב דמוי שפת תכנות, כתיב מתמטי וכתיב חפשי.

מטרת השימוש בקוד דמה היא ליצור **תיאור חד משמעי** של האלגוריתם אשר מאפשר יישום האלגוריתם באמצעות תוכנה.

אנו מניחים כי קורא מיומן, יכול לעקוב אחר ביצוע האלגוריתם באופן חד משמעי.

להלן נסקור את האלמנטים העיקריים המשמשים בקוד דמה:

© כל הזכויות שמורות לפרופסור עמוס ישראלי

- **כותרת האלגוריתם** – זוהי השורה הראשונה בקוד והיא מכילה את שם האלגוריתם ואת הקלט אתו האלגוריתם מתבצע. כותרת האלגוריתם אינה מכילה הוראות לביצוע.
- **הוראת השמה** – הוראת ההשמה נראית כך : $x \leftarrow exp$. בתיאור זה, x הוא משתנה ו-exp הוא ביטוי אריתמטי או בוליאני, המכיל מספרים, משתנים וסימני פעולות, כך שערכו של exp מוגדר. ביצוע הוראה זו מאחסן את ערכו של exp, במשתנה x.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

- **הוראות בקרה** – הוראות בקרה

מבוטאות בעזרת מבנים מקובלים:

1. לתיאור טסטים נשתמש במבנה **if (*cond*) else**.
2. לתיאור לולאות נשתמש בכל אחד מן המבנים הבאים, המקובלים בשפות תכנות:

- **for … end for**
- **while(*cond*) … end while**
- **do … while (*cond*)**

בהוראות אלה *cond* הוא ביטוי בוליאני שערכו הוא *true* או *false*.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

- **הוראת *return*** – הוראת ***return***

  נראית כך: ***return***(...). הוראה זו, מציינת את סיום ביצוע האלגוריתם והחזרת הפלט, המופיע בין הסוגריים. לעתים, האלגוריתם אינו מחזיר פלט ואז הסוגריים מושמטים.

- **סיום האלגוריתם** – לכל אלגוריתם, *Alg*, השורה האחרונה של *Alg* היא **end**(*Alg*). שורה זו אינה מכילה הוראות לביצוע.

- **קריאה לאלגוריתם מוכר** – בכל שלב באלגוריתם אפשר לשלב בקוד קריאה לאלגוריתם אחר שכבר

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוגדר. לדוגמה ההוראה

$y \leftarrow Examp(j)$, קובעת את ערכו של המשתנה y, לערך המחושב על ידי האלגוריתם *Examp* המופעל עם הארגומנט j.

- **תיאור פעולה בשפה טבעית** – אלגוריתם יכול לכלול כל תיאור של פעולה או של סדרת פעולות בשפה טבעית בתנאי שהתיאור הוא חד-משמעי.

תיאור של פעולה הוא **חד משמעי** אם אפשר להביע את הפעולה בתוכנה רק באופן יחיד.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה: אלגוריתם לחישוב סכום של מערך

באיור 1.4 מוצג אלגוריתם *SumArray* אשר פותר את הבעיה **סכום של מערך:**

```
SumArray(A,n)
    sum ← 0
    for i ← 1 to n
        sum ← sum + A[i]
    endfor
    return(sum)
end(SumArray)
```

איור 1.4: אלגוריתם לחישוב סכום של מערך

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שימו לב

הארגומנט n מייצג את מספר האיברים במערך הקלט A. בדרך כלל, לא נציין את המספר הזה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מעקב אחר ביצוע אלגוריתם

המעקב הוא **הרצה יבשה** של אלגוריתם (או של קטע ממנו). המעקב מהווה פעולה בסיסית שמשתמשים בה פעמים רבות במהלך הוכחות נכונות אלגוריתמים.

לדוגמה נתבונן באלגוריתם ***Dugma***, המוצג באיור 1.6.

```
Dugma(v)
    x ← v
    y ← 2
    z ← x + y
    return(z)
end(Dugma)
```

איור 1.6: האלגוריתם *Dugma*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מעקב אחרי ביצוע אלגוריתם *Dugma* נראה כך:

- המשתנה x מקבל את הערך השמור בארגומנט v.
- המשתנה y מקבל את הערך 2.
- האלגוריתם מחבר את המשתנים x ו-y, ומאחסן את תוצאת החיבור במשתנה z.
- ערך המשתנה z נשלח לפלט.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

איך לפתח אלגוריתם

פיתוח אלגוריתם דומה למדי לפיתוח תכנה. כשמתקבלת בעיה חישובית, מנסים לחשוב איך לפתור אותה בעזרת עט ונייר. בדרך כלל עולה רעיון כלשהו, ואז מנסים להביא את הרעיון על ידי אלגוריתם.

כאשר הגירסה הראשונה של האלגוריתם מוכנה, אפשר לבצע הרצות "יבשות", **באמצעות מעקב**, ולוודא שהאלגוריתם עובד.

אם מגלים שהאלגוריתם שגוי, מתקנים אותו, וממשיכים בהרצות יבשות, וזאת עד שאנו משוכנעים רמת ביטחון גבוהה יחסית שהאלגוריתם שלנו נכון.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הרצות ניסיון

במקרה שהאלגוריתם מסובך, אפשר לתכנת אותו ולבצע הרצות ניסיון:

1. בוחרים מספר קלטים לדוגמה. כל קלט כזה, צריך לייצג בעיה פוטנציאלית באלגוריתם.

   לדוגמה:

   - מערך עם איבר יחיד
   - מערך עם איברים שליליים.

2. מריצים את האלגוריתם על כל אחד מן הקלטים לדוגמה ומוודאים באופן בלתי תלוי שהאלגוריתם מניב פלט נכון.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

3. במקרה שהאלגוריתם המוצע אינו נכון, יש סיכוי טוב שהרצת הניסיון תניב תוצאה שגויה ובכך תחסוך את המאמץ בהוכחת אלגוריתם שגוי.
4. זיהוי השגיאה בדרך כלל מסייע בתיקון הפגם באלגוריתם.

לאחר כתיבת האלגוריתם אפשר לגשת להוכחת הנכונות שלו. בשקפים הבאים נגדיר מהי נכונות אלגוריתם, ואיך מוכיחים נכונות כזאת.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 1.45: נכונות של אלגוריתם

אלגוריתם Al לפתרון בעיה חישובית P $P=\langle In_P, Out_P \rangle$ הוא **נכון** אם ביצוע של Al על **כל** קלט $i \in In_p$, **מסתיים** עם הפלט $Out_P(i)$.

שימו לב:

1. לכל קלט חוקי I (כל איבר בקבוצת הקלטים הוא **קלט חוקי**) חישוב האלגוריתם עם i מסתיים.
2. אם קיים אפילו קלט יחיד עבורו האלגוריתם נכשל בחישוב הפלט, האלגוריתם הוא **שגוי**.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת נכונות של אלגוריתם

נניח שקיבלנו קטע של קוד דמה ונטען שזהו אלגוריתם לפתרון בעיה P. איך נוכל לוודא שהטענה הזו נכונה?

הרעיון הראשון שעולה הוא לממש את האלגוריתם באמצעות שפת התכנה החביבה עלינו, להריץ אותו עם קלטים שונים מן הקבוצה, ולבדוק האם הפלטים המתקבלים נכונים. לצערנו, השיטה הזו **לא יכולה להניב הוכחה**.

הסיבה היא שקבוצת הקלטים היא אין סופית ואילו השיטה שהצענו, מתאימה רק לבדיקת מספר סופי של קלטים.

לכן נוכיח נכונות של אלגוריתמים בדרכים מתמטיות, כפי שמפורט להלן.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת נכונות של אלגוריתמים

נניח כי $P =< In_P, Out_P >$ היא בעיה חישובית ויהי Al אלגוריתם לפתרון P.

כדי להוכיח את נכונותו של Al יש להוכיח:

1. **סיום** - לכל קלט $i \in In$, חישוב Al על i **מסתיים** לאחר ביצוע מספר צעדים סופי.
2. **נכונות פלט** - לכל קלט $i \in In$, חישוב Al על i מסתיים עם הפלט $out_P(i)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת סיום

אלגוריתם **איטרטיבי** הוא אלגוריתם המכיל לולאות והוראות **if**, אך אינו מבצע קריאות רקורסיביות.

כדי להוכיח סיום של אלגוריתם איטרטיבי, מספיק לבדוק:

1. מספר המעברים בכל אחת מן הלולאות הוא חסום.
2. מספר צעדי החישוב בכל מעבר בלולאה גם הוא חסום.

לדוגמה נוכיח כי אלגוריתם *SumArray* מסתיים

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 1.5: סיום אלגוריתם

SumArray

כל ביצוע של *Sum* עם מערך מספרים A מסתיים.

```
SumArray(A,n)
    sum ← 0
    for i ← 1 to n
        sum ← sum + A[i]
    endfor
    return(sum)
end(SumArray)
```

הוכחה

מן הקוד נובע שמספר המעברים בלולאת האלגוריתם שווה למספר איברי A . מספר צעדי החישוב בכל מעבר בלולאה הוא קבוע. **מש"ל**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה לשיטה שעצירתה לא הוכחה

בדרך כלל, הוכחת סיום היא פשוטה ולא נתעכב עליה.

במקרים נדירים הוכחת הסיום היא קשה מאוד. לדוגמה, באיור 1.5 מוצגת השיטה *phi*. זוהי שיטה פשוטה ביותר. לכל ערך נתון של הארגומנט *n*, אפשר להריץ את השיטה ולכל קלט שנבחר הביצוע תמיד הסתיים, אך עד כה לא הוכח כי השיטה עוצרת לכל המספרים הטבעיים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

```
phi(n)
   do forever
     if (n=1) return()
     if (n|2) n ← n/2
           else n ← 3n+1
      endif
    end do
end(Phi)
```

איור 1.5: הפונקציה *phi*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בניית משפט נכונות – תבנית הטענה

השלב הראשון בהוכחת הנכונות הוא בניית **משפט נכונות**.

כל משפט נכונות נבנה לפי התבנית הבאה:

תבנית משפט נכונות

לכל קלט $i \in In_P$, חישוב Alg עם i מסתיים עם הפלט $out_P(i)$.

איור 1.7: תבנית משפט הנכונות

כדי להפוך את התבנית למשפט נכונות מלא, יש **להציב**:

1. שם האלגוריתם **במקום** Alg.
2. תיאור מדויק של הקלט **במקום** i.
3. תיאור מדויק של הפלט **במקום** $out_p(i)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמא - בניית משפט נכונות עבור

אלגוריתם *SumArray*

משפט הנכונות מתקבל מהצבת תיאורי הקלט והפלט בתבנית:

לכל **מערך מספרים** A (תיאור הקלט),

האלגוריתם *SumArray*

(שם האלגוריתם)

מחזיר את **סכום איברי המערך** A

(תיאור הפלט).

משפט הנכונות תלוי אך ורק בבעיה החישובית שהאלגוריתם פותר.
ההתייחסות היחידה לאלגוריתם היא הזכרת שם האלגוריתם.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת נכונות פלט

כדי להוכיח נכונות של אלגוריתם *Alg* עלינו להבין את דרך פעולתו של *Alg*. הכוונה היא לנסות ולהבין את הקשר בין הארגומנטים (פרמטרים) הנמסרים לאלגוריתם ובין המשתנים המופיעים בקוד, את תפקידם של המשתנים המופיעים בקוד ואת הדרך בה מחושב הפלט.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה

נראה את הדרך בה אנו מבינים את הקשר בין מרכיבי האלגוריתם תוך שימוש במעקב אחרי ביצוע אלגוריתם *SumArray*:

```
SumArray(A,n)
    sum ← 0
    for i ← 1 to n
        sum ← sum + A[i]
    endfor
    return(sum)
end(SumArray)
```

בסיום האלגוריתם, הפלט מאוחסן במשתנה *sum* .

© כל הזכויות שמורות לפרופסור עמוס ישראלי

איך האלגוריתם מחשב את ערכו של sum?

אפשר לראות, על ידי מעקב, כי

- בסיום המעבר הראשון בלולאת האלגוריתם, לפני ההוראה **endfor**, $sum = A[1]$.
- בסיום המעבר השני, $sum = A[1] + A[2]$.
- וכן הלאה.

אם נמשיך בהיגיון זה, נגיע למסקנה כי בסיום המעבר ה-i בלולאה, ערכו של המשתנה sum שווה לסכום i האיברים הראשונים במערך הקלט.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

$$sum = A[1] + \ldots + A[i]$$

מכאן אפשר להסיק כי בסיום המעבר ה-n בלולאת האלגוריתם, ערכו של המשתנה sum שווה לסכום כל אברי המערך.

$$sum = A[1] + \ldots + A[n]$$

הוכחת הנכונות משתמשת בהיגיון זה. לדוגמה עבור מערך עם שני איברים ההוכחה תראה כך:

- בתחילת האלגוריתם ערכו של המשתנה Sum מאותחל ל-0.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

- בסיום המעבר הראשון בלולאה ערכו של המשתנה *Sum* שווה ל-$A[1]$.
- בסיום המעבר השני בלולאה ערכו של המשתנה *Sum* שווה ל-$A[2]+A[1]$ ומשתנה זה מוחזר כפלט.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

עבור מערך עם 1,000,000 איברים ההוכחה תראה כך:

- בתחילת האלגוריתם ערכו של המשתנה *Sum* מאותחל ל-0.
- בסיום המעבר הראשון בלולאה ערכו של המשתנה *Sum* שווה ל-$A[1]$.
- ...(עוד 999,998 שורות כלה
- בסיום המעבר ה-1,000,000 בלולאה ערכו של המשתנה *Sum* שווה ל-$A[1]+\ldots+A[1,000,000]$

בדרך זו אפשר להוכיח את נכונות האלגוריתם עבור כל המערכים שמספר האיברים בהם הוא **<u>חסום.</u>**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שיטת **ההוכחה באינדוקציה** מאפשרת הוכחה שאורכה קבוע עבור כל מערכי המספרים האפשריים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת נכונות של אלגוריתמים איטרטיביים (עם לולאה)

נכונות אלגוריתם איטרטיבי (אלגוריתם המכיל לולאה) מוכיחים **באינדוקציה על מספר המעברים בלולאת האלגוריתם.**

לדוגמה נוכיח כעת את נכונות אלגוריתם *SumSrray*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 1.8: משפט הנכונות לאלגוריתם *SumArray*

אלגוריתם *SumArray* מקבל כקלט מערך מספרים A מחזיר **סכום איברי המערך A**.

הוכחה

נוכיח באינדוקציה על מספר המעברים בלולאה את הטענה הבאה:

לכל i, $(i=1,2,\ldots)$ כאשר בקרת האלגוריתם מגיעה לסוף הלולאה בפעם ה-i, מתקיימת שמורת הלולאה הבאה:

$$sum = A[1]+\ldots+A[i]$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שימו לב:

שמורת הלולאה היא ביטוי בוליאני (מקבל את הערך **true** או **false**) המבטא קשר לוגי בין ערכי הקלט, משתני האלגוריתם והערך (או הערכים) המוחזרים בפלט.

שמורת הלולאה מקבלת את הערך **true** בכל פעם שבקרת האלגוריתם מגיע לסיום הלולאה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחה (באינדוקציה על מספר המעברים בלולאה)

בסיס

עלינו להוכיח כי כאשר בקרת האלגוריתם מגיעה לסוף הלולאה בפעם הראשונה מתקיים $sum = A[1]$.

הוכחת הבסיס

ההוכחה היא באמצעות מעקב אחרי ביצוע האלגוריתם.

תרגיל

כתבו הוכחה מדויקת המשתמשת במעקב ומוכיחה שבסיום המעבר הראשון בלולאה הטענה מתקיימת.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

לנוחיותכם נציג שוב את האלגוריתם

```
SumArray(A,n)
    sum ← 0
    for i ← 1 to n
        sum ← sum + A[i]
    endfor
    return(sum)
end(SumArray)
```

צעד האינדוקציה

נניח כי לאחר i מעברים בלולאה מתקיים:

$$sum = A[1] + \ldots + A[i]$$

במהלך המעבר ה-$i+1$ בלולאה, מחברים למשתנה sum את $A[i+1]$.

מכאן ומהנחת האינדוקציה נובע כי

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בתום המעבר ה-$i+1$ בלולאה מתקיים:

$$sum = \underbrace{\{A[1]+...+A[i]\}}_{InductionHypothsys} + \underbrace{A[i+1]}_{Tracking}$$

$$= A[1]+\ldots A[i+1]$$

כאשר האיבר הראשון נובע מהנחת האינדוקציה והאיבר השני נובע ממעקב.

מש״ל

<u>הוכחת המשפט</u>

לולאת האלגוריתם מתבצעת n פעמים. נציב n בשמורה ונקבל שבסיום המעבר ה-n בלולאת האלגוריתם מתקיים:

$$sum = A[1]+\ldots+A[n]$$

מש״ל

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שיטת השמורה בארבעה שלבים

נתאר כעת בפרוט את שלבי שיטת ההוכחה באינדוקציה על מספר המעברים בלולאה.

שלב 1: ניסוח השמורה

מנסחים טענה אודות הקשר בין ערכי הקלט, ערכי משתני האלגוריתם, מספר המעברים בלולאה, ודורשים שהטענה הזו תתקיים (תהיה נכונה) בכל פעם שהאלגוריתם מגיע לנקודה מסוימת בלולאה. בנוסף נדרש שנכונות האלגוריתם תהווה תוצאה של נכונות השמורה במעבר האחרון בלולאה.
לטענה קוראים **שמורת הלולאה**, ולנקודה שבה השמורה מתקיימת קוראים **נקודת השמורה.**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

<u>שלב 2: הוכחת בסיס האינדוקציה</u>

מוכיחים על ידי מעקב אחרי ביצוע האלגוריתם כי כאשר בקרת האלגוריתם מגיעה לנקודת השמורה בפעם הראשונה, השמורה מתקיימת.

<u>שלב 3: שלב האינדוקציה</u>

מניחים כי השמורה נכונה כאשר בקרת האלגוריתם מגיעה לנקודת השמורה במעבר ה- i, ומוכיחים, **שוב בעזרת מעקב,** כי השמורה נכונה כאשר בקרת האלגוריתם מגיעה לנקודת השמורה בתום המעבר ה- $i+1$.

<u>שלב 4: סיום ההוכחה</u>

הוכחת המשפט תתקבל מהצבת מספר המעברים בלולאה, n בשמורה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שמורות חליפיות

בדרך כלל, אפשר לבחור כמה שמורות חלופיות. לכל שמורה חלופית תתאים נקודת שמורה משלה:

לדוגמא: אפשר גם לבחור כשמורה את הטענה בתחילת המעבר ה- i בלולאת האלגוריתם, ($1 \le i \le n$), **לפני ביצוע הוראת ההשמה** מתקיים:

$$sum = A[1] + \cdots + A[i-1]$$

בתרגיל 1.1 אתם מתבקשים להוכיח את נכונות האלגוריתם *Sum* בעזרת השמורה החליפית.

באופן כללי, כדאי לבחור את השמורה בהוכחה, לפי פשטות ההוכחה המתקבלת.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שמורות גרועות

יש אינסוף ביטויים אשר הם נכונים בכל פעם שבקרת התכנה מגיעה לנקודת השמורה, אך הם חסרי תועלת להוכחת הנכונות.

דוגמה לשמורה כזו היא

$$1+2=3.$$

זה ביטוי נכון, אך הוא אינו קושר בין הקלט לבין משתני התכנית או בין משתני התכנית לבין הפלט.

עוד שמורה גרועה היא

$$1 \le i \le n$$

בחירת השמורה הנכונה היא האתגר שבהוכחת הנכונות.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיבוכיות האלגוריתם

להשלמת הטיפול באלגוריתם *Sum* נציג את ניתוח הסיבוכיות שלו:

1. הלולאה מתבצעת בדיוק n פעמים.
2. בכל מעבר מתבצע מספר פעולות קבוע (Constant), כלומר מספר פעולות שאינו תלוי בגודל הקלט.

משתי נקודות אלה נובע כי הסיבוכיות היא לינארית, כלומר $\Theta(n)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

האלגוריתם *Max*: חישוב איבר מרבי במערך חד ממדי

נתון האלגוריתם *Max*

```
Max(A)
    M ← A[1]
    for i = 2 to n
        M ← max(M, A[i])
    endfor
    return(M)
end(Max)
```

איור 1.10: אלגוריתם *Max*

בתרגיל 1.2 תתבקשו לנסח את הבעיה החישובית שהאלגוריתם פותר, להוכיח את נכונותו, ולנתח את הסיבוכיות שלו.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

האלגוריתם *Copy*: העתקת מערך חד ממדי

נתון האלגוריתם *Copy*

```
Copy(A)
    for i ← 1 to length(A)
        C[i] ← A[i]
    endfor
    return(C)
end(Copy)
```

איור 1.11: אלגוריתם *Copy*

שימו לב: אלגוריתם *Copy* משתמש במערך C ללא הגדרה. נזכור שזהו קוד דמה. כל זמן שהסימונים מובנים, אין צורך בפורמליסטיקה מיותרת.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בתרגיל 1.3 תתבקשו לנסח את הבעיה החישובית שהאלגוריתם $Copy(A)$ פותר, להוכיח את נכונותו, ולנתח את הסיבוכיות שלו.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

טיפול בלולאות מקוננות

עד כה התווינו את דרך הטיפול באלגוריתמים שיש בהם לולאה אחת. כעת נעבור להדגמת טיפול באלגוריתם $Max2d$ המכיל שתי לולאות מקוננות. אלגוריתם זה פותר את הבעיה החישובית ״חישוב איבר מרבי במערך דו-ממדי המוצגת באיור 1.12 :

קלט

מערך דו ממדי A שאיבריו מספרים.

פלט

איבר מרבי במערך A.

איור 1.12: הבעיה החישובית ״חישוב איבר מרבי במערך דו-ממדי״

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם לחישוב איבר מכסימלי במערך ד ממדי

```
Max2d(A)
    for i = 1 to n
        for j = 1 to n
            Line[j] ← A[i, j]
        endfor
        B[i] ← Max(Line)
    endfor
    return(Max(B))
end(Max2d)
```

איור 1.13: אלגוריתם *Max2d*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם *Max2d* - תיאור

נסמן את השורה ה-i של המערך A ב-$A[i,1...n]$. האלגוריתם מבצע n איטרציות (מעברים בלולאה):

באיטרציה ה-i האלגוריתם מחשב את האיבר המרבי בשורה ה-i ומכניס איבר זה אל $B[i]$ לאחר מכן האלגוריתם מפעיל את השיטה $Max(B)$ כדי לחשב את האיבר המרבי במערך B.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת נכונות

אחת הדרכים להוכחת נכונות האלגוריתם היא להוכיח נכונות של אלגוריתם אחר השקול לאלגוריתם המקורי.

נתבונן באלגוריתם 1.14 :

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם שקול לחישוב איבר מכסימלי

```
Max2d2(A)
    for i = 1 to n
        Line ← Copy(A[i,1...n])
        B[i] ← Max(Line)
    endfor
    return(Max(B))
end(Max2d2)
```

אלגוריתם 1.14: האלגוריתם *Max2d2*

שימו לב: באלגוריתם זה, הסימון $A[i,1...n]$ מייצג את השורה ה-i במערך הקלט A.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם שקול לחישוב איבר מכסימלי

בתרגיל 1.4 תתבקשו לנסח את הבעיה החישובית שהאלגוריתם $Max2d2$ פותר, להוכיח את נכונותו, ולנתח את הסיבוכיות שלו.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

נספח: ניתוח זמן ריצה (חזרה)

זמן ריצה הוא הגורם החשוב ביותר שיש לבחון כאשר בוחרים אלגוריתם לפתרון בעיה חישובית כלשהי.

לאחר שהשתכנענו שאלגוריתם הוא נכון, יש לנתח את **סיבוכיות הזמן** שלו.

סיבוכיות הזמן של אלגוריתם, היא פונקציה מתמטית המאפשרת לנו לחזות את זמן הריצה של תוכנית המבוססת על האלגוריתם ולהשוות בין אלגוריתמים שונים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרת סיבוכיות הזמן של אלגוריתמים

בתחילת הדיון נמנה את הגורמים העיקריים המשפיעים על זמן הביצוע של תכנית לפתרון בעיה נתונה:

1. יעילות הקידוד. (תרגום מאלגוריתם לתכנית).
2. יעילות המהדר. (תרגום משפת על לשפת מכונה).
3. יעילות המעבד.
4. יעילות האלגוריתם.

האם אתם מכירים גורם נוסף?

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מה מידת ההשפעה של כל גורם?

1. יעילות הקידוד - כל שורה משורות האלגוריתם תקודד למספר **קבוע** של הוראות בשפה עילית.

יעילות המהדר - כל הוראה בשפה עילית תתורגם למספר **קבוע** של הוראות בשפת מכונה. **ללא תלות באורך הקלט**

מהדר יעיל יותר יכול להקטין מספר זה **בגורם קבוע** כלשהו.

3. הגדלת מהירות המעבד -

(לדוגמא מ 1 MHz לו 1 ל-GHz ,

תקטין את זמן הריצה בגורם קבוע (בדוגמא 1000).

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שיפורים בגורם קבוע

כל שיפור באיכות ה**קידוד המהדר** או **המעבד**, משפיע על זמן הריצה **אך ורק** על ידי הכפלה בקבוע, שאינו תלוי באורך הקלט.

אין לזלזל בשיפור שיקטין את זמן הריצה למחצית, לשליש, לעשירית, או אף לאלפית (כאשר משפרים מחשב) מערכו הקודם. כאשר מתמודדים עם בעייה קשה, יש למצות את כל אפשרויות השיפור של הפתרון.

לעתים כל האפשרויות גם יחד אינן מספיקות בהתמודדות עם בעיות בגודל (אורך קלט) הולך וגדל.

האם יש אפשרויות שיפור נוספות?

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שיפור האלגוריתם

מספר הצעדים שאלגוריתם מסויים מבצע הוא **פונקציה של אורך הקלט לאלגוריתם.** במהלך הקורס אנו נוכח כי אפשר לשפר אלגוריתמים מסויימים בסדר גודל, למשל מסיבוכיות n^2 לסיבוכיות $n\log n$. ברור אם כן כי בעזרת שיפור האלגוריתם אפשר לשפר את זמן הריצה בגורם **העולה על קבוע. בשום דרך אחרת אי אפשר להשיג שיפור כזה.**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיבוכיות זמן ריצה

נתון אלגוריתם A הפותר בעיה P.

נתבונן בסדרת קלטים מ-P

$$I_1, I_2, \ldots, I_n, \ldots$$

שאורכם הולך וגדל.

בדרך הטבע, ככל שאורך הקלט גדול יותר, האלגוריתם יעבוד זמן רב יותר.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרת סיבוכיות הזמן

סיבוכיות הזמן של האלגוריתם או בפשטות **זמן הריצה** של האלגוריתם תוגדר כפונקציה המתאימה את מספר צעדי האלגוריתם לאורך הקלט.

$t(n) =$ ***מספר צעדים מירבי בריצה של האלגוריתם על קלט באורך*** n

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיבוכיות המקרה הגרוע ביותר

יש אלגוריתמים שזמן הריצה שלהם על קלטים באורך שווה משתנה מקלט לקלט.

דוגמה: במיון מהיר (Quick-Sort), יש קלטים רבים באורך n, שזמן הריצה שלהם הוא $\Theta(n \log n)$ אך עבור קלט **ממוין,** זמן הריצה הוא $\Theta(n^2)$.

במקרה זה, עלינו לבחור מהו זמן הריצה המייצג את הקלטים באורך n.

בדרך כלל, מקובל לבחור עבור כל n את הקלט עבורו האלגוריתם רץ בזמן הארוך ביותר (Worst Case).

© כל הזכויות שמורות לפרופסור עמוס ישראלי

נימוקים לשימוש בWorst Case

הנימוק המרכזי הוא **מניעת הפתעות.**

כשמשתמשים ב Worst Case,

סיבוכיות האלגוריתם היא **חסם עליון**

על זמן הריצה עבור כל המקרים.

נימוק חשוב נוסף הוא שיחסית קל

לחשב את המקרה הגרוע ביותר.

במקרים מסוימים (כגון במיון מהיר)

נתיחס גם ל**זמן הריצה הממוצע** (יוגדר

מאוחר יותר) .

שימו לב: לעיתים משתמשים במושג

Best Case לצרכי הסבר.

אין משמעות לסיבוכיות המקרה הטוב ביותר ואין להשתמש במושג זה לצרכי ניתוח זמן ריצה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

<u>הגדרה פורמלית של סיבוכיות זמן</u>

יהי Al אלגוריתם כלשהו. לכל קלט חוקי I נסמן ב $t_{Al}(I)$ את זמן הריצה של Al על הקלט I.

סיבוכיות הזמן של Al מוגדרת על ידי:

$$t_{Al}(n) = \max_{|I|=n} \{t_{Al}(I)\}$$

לכל $n > 0$. כאשר המכסימום נלקח על קבוצת כל הקלטים באורך n.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

ניתוח זמן ריצה

מטרתו של ניתוח זמן ריצה של אלגוריתם היא:

1. לסייע לנו **לצפות** את זמן הריצה של תכנית המבוססת על האלגוריתם ללא הרצה של התוכנית.
2. להשוות יעילות של כמה אלגוריתמים.
3. להחליט אם לחפש אלגוריתם נוסף.

שימו לב: מפני שאלגוריתם הוא ברמת הפשטה גבוהה, לעתים קשה לעמוד על משמעות שינוי בגורם קבוע.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מסקנה

בדיון בסיבוכיות זמן של אלגוריתמים נבדוק רק שיפורים שיקטינו את **סדר הגודל** של זמן הריצה **ונתעלם מגורמים קבועים.**

דוגמא

נניח כי אלגוריתם A פותר בעיה מסוימת בזמן n^2+7 ואלגוריתם B פותר אותה הבעיה בסיבוכיות n^2+12

אנו נאמר כי לשני האלגוריתמים סיבוכיות זמן $\Theta\left(n^2\right)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

השוואת קצבי גידול

הסימונים O, Θ, וΩ מאפשרים לנו להשוות בין קצבי הגידול של פונקציות שונות כאשר n שואף לאינסוף.

דוגמא

נניח כי זמן הריצה של תוכנית כלשהי מבוטא על ידי הפונקציה $T(n)$ המוגדרת על ידי:

$$T(n) = 31n^3 + 17.5n^2 - 3.2n + 1077$$

קל לראות (איך?) כי כאשר $n \to \infty$ ערך $T(n)$ נשלט על ידי n^3. במקרה כזה נאמר כי $T(n) = \Theta(n^3)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרות

נאמר כי $f(n) = O(g(n))$ (הפונקציה $g(n)$ **חוסמת מלמעלה באופן אסימפטוטי** את הפונקציה $f(n)$) אם קיימים קבועים c_0 ו n_0 המקיימים:

לכל $n > n_0$, $f(n) \leq c_0 g(n)$.

נאמר כי $f(n) = \Omega(g(n))$ (הפונקציה $g(n)$ **חוסמת מלמטה באופן אסימפטוטי** את הפונקציה $f(n)$) אם קיימים קבועים c_1 ו n_1 המקיימים:

לכל $n > n_1$, $f(n) \geq c_1 g(n)$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שימו לב: אם $f(n)=O(g(n))$ אז $g(n)=\Omega(f(n))$.

<u>המשך ההגדרות</u>

הפונקציה $f(n)$ היא $\Theta(g(n))$ (הפונקציה $f(n)$ **שוה אסימפטוטית** לפונקציה $g(n)$) אם

1. $f(n)=O(g(n))$

וגם

2. $g(n)=O(f(n))$

שימו לב: אם $f(n)=\Theta(g(n))$ אז גם $g(n)=\Theta(f(n))$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמא

תהי

$$T(n)=31n^3+17.5n^2-3.2n+1077$$

נראה כי $T(n)$ היא $O(n^3)$.

$$T(n)\le 31n^3+17.5n^2+1077$$

עבור $n>1077$ נקבל כי

$$T(n)\le 31n^3+n^3+n^3=33n^3$$

משי״ל.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סקלה למדידת קצב הגידול של פונקציות

אנו מחפשים דרך לתאר התנהגות של פונקציות. הדרך שנבחרה היא להשוות כל פונקציה לקבוצת פונקציות בסיסיות המוכרות לכל.

הפונקציות הבסיסיות הן:

1. פונקציות מעריכיות: $f(n)=a^{n^k}$, $k=1,2,\ldots$.
2. מונומים $f(n)=n^k$, $k=1,2,\ldots$
3. פונקציות לוגריתמיות $f(n)=(\log n)^k$, $k=1,2,\ldots$.

שימו לב: כל הפונקציות הללו הן **חיוביות**.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הסדר האסימפטוטי

להלן הסדר האסימפטוטי של כמה פונקציות מוכרות:

$$1 < \log n < (\log n)^k < \log n^{k+1}$$
$$< n^{\varepsilon}\,(\forall \varepsilon > 0) < n^{1/2} <$$
$$n < n\log n < n^k < n^{k+1} <$$
$$< 2^n < n!$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

השוואת סדרי גודל

בהינתן פונקציה כלשהי נרצה להשוות אותה לאחת הפונקציות התקניות.

ברובם המכריע של המקרים איננו צריכים לחשב גבולות כלל אלא להשתמש בהגיון ובכמה חוקים בסיסיים.

כדוגמא נביא שאלה מתוך בוחן:

קבע את יחסי סדר הגודל בין

$$f(n) = n(\log n)^2$$

לבין

$$g(n) = \frac{n^2}{\log n}$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בדוגמא זו, קל מאוד לראות, למשל על ידי הכפלת שתי הפונקציות בפונקציה $g(n)$ כי $\log n$ גדלה מהר יותר מ $f(n)$ כלומר $f(n) = O(g(n))$.

שימו לב:

לעתים אנו אומרים כי סיבוכיות האלגוריתם היא $O(f(n))$ כאשר בעצם אנו מתכוונים לומר $\Theta(f(n))$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמא נוספת

נתון כי הפונקציה $f(n)$ מתנהגת באופן שונה עבור ערכי n שונים כמתואר בטבלה הבאה:

n	0-25	26-250	>250
$f(n)$	n^4	n^3	n^2

מהי סיבוכיות הפונקציה f?

תשובה: מן ההגדרות נובע מייד כי מה שקובע את סיבוכיות הפונקציה היא התנהגותה כאשר ערכי n שואפים לאינסוף. להתנהגות f עבור מספר סופי של ערכי n אין שום חשיבות ולכן

$$f(n)=\Theta\left(n^2\right)$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיכום

בהצאה זו הגדרנו והדגמנו את המושגים הבסיסיים שישמשו אותנו במהלך הקורס והם:

1. בעיה חישובית.
2. אלגוריתם.
3. נכונות אלגוריתם.
4. סיבוכיות אלגוריתם.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תרגילים

תרגיל 1.1: נתונה טענה:

$$sum = \sum_{j=1}^{i-1} A[j]$$

זהו נקודה בלולאה שבאלגוריתם *Sum* (אלגוריתם 1.7) שבה הטענה מהווה שמורה. השתמשו בשמורה הנתונה, כדי להוכיח את נכונות האלגוריתם.

תרגיל 1.2: נסחו את הבעיה החישובית שהאלגוריתם *Max* פותר.

הוכיחו את נכונות האלגוריתם *Max*, באינדוקציה על מספר המעברים בלולאה ונתחו את סיבוכיות האלגוריתם.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

<u>תרגיל 1.4:</u> נסחו את הבעיה החישובית שהאלגוריתם $Max2d2$ פותר.
הוכיחו את נכונות האלגוריתם באינדוקציה על מספר המעברים בלולאה ונתחו את סיבוכיות האלגוריתם.

© כל הזכויות שמורות לפרופסור עמוס ישראלי